

Java Programming

A practical study reference with foundational concepts, examples, and common review questions.

1. Platform and Types

Java source is compiled by `javac` into bytecode; a Java Virtual Machine (JVM) executes it and may use just-in-time compilation. A JDK provides development tools and runtime components, with packaging varying by vendor. Java is statically typed; primitive types hold values, while reference types refer to objects.

Packages organize names and APIs. Build tools such as Maven and Gradle manage dependencies, builds, and tests. A conventional program entry point is `public static void main(String[] args)`.

2. Classes and Interfaces

A class defines fields, constructors, and methods. Encapsulation protects invariants; interfaces define contracts; inheritance specializes an is-a relationship. Prefer composition when it better represents collaboration. Use private fields and expose deliberate public methods.

Overloading uses the same method name with different parameters; overriding supplies a subtype implementation. Use `@Override`. static members belong to the class; final can prevent reassignment, overriding, or inheritance depending on its use.

3. Generics and Collections

Generics provide compile-time type safety. List stores an ordered sequence, Set stores unique values, and Map associates keys with values. ArrayList is a common resizable list; HashMap offers expected constant-time lookup under typical hashing assumptions, but does not promise iteration order.

Choose collections by ordering, uniqueness, and lookup needs. Wildcard bounds such as `? extends T` and `? super T` express safe producer and consumer use. Streams process values with operations such as filter and map; terminal operations trigger evaluation.

4. Errors and Concurrency

Checked exceptions must be caught or declared; unchecked exceptions extend RuntimeException. Catch specific failures. Try-with-resources closes AutoCloseable resources. ExecutorService manages task execution; protect shared mutable state with synchronization, locks, atomics, or concurrent collections.

Common questions: the JVM executes bytecode while a JDK supplies development tools; encapsulation protects an object's state; an interface lets multiple types satisfy one contract. Strong typing does not prevent injection or authorization bugs: use parameterized SQL and least privilege.